

Android Application for Collaborative Trip Planning

Ilya Novik
Faculty of Computer Science
Higher School Of Economics
Moscow, Russia
iinovik@edu.hse.ru

Abstract—Group travel planning is typically fragmented across chat apps, map tools, booking services, and separate expense trackers. This fragmentation creates duplicated work, unclear decisions, delays, and budget errors. The proposed project addresses this gap by implementing an Android application and backend platform that unify collaborative trip editing, idea discussion, itinerary building, expense balancing, weather support, and AI-generated activity suggestions. As a project-oriented graduation thesis, the contribution is practical engineering value rather than scientific novelty: system integration quality, maintainable architecture, and production readiness. The solution uses a Kotlin Android client and a Kotlin Ktor backend with PostgreSQL, real-time comment channels, and offline synchronization. AI functionality is powered by Yandex GPT, and the production backend is deployed on Yandex.Cloud infrastructure. The platform is designed for practical operation in student and real-user scenarios, including role-governed collaboration and predictable release processes. The expected outcome is a deployable product that reduces coordination overhead for small travel groups and is ready for publication in Google Play.

Keywords—AI suggestions, Android application, collaborative trip planning, expense sharing, itinerary management, Ktor, offline synchronization, PostgreSQL

I Introduction

Planning a group trip is usually treated as a simple organizational task, but in practice it is a distributed coordination problem. Participants discuss destinations in messengers, store ideas in notes, build routes in map services, track budgets in dedicated finance apps, and check weather in separate tools. This multi-tool workflow causes information loss and inconsistent decisions. The owner of a trip often becomes a manual integrator of scattered data, while other participants have limited visibility into up-to-date plans. As a result, teams spend time on synchronization instead of planning quality.

The project is designed to solve this practical gap by delivering a single mobile workflow for collaborative planning. The target platform is Android, with backend support for multi-user data consistency and secure access. Functionally, the product combines authorization, trip lifecycle management, invite-based team collaboration, idea discussion, itinerary management

by days, shared expenses with balance calculation, weather retrieval, and AI-assisted activity suggestions.

As a project-oriented thesis, the core contribution is technical and applied: architecture design, integration of multiple subsystems, and implementation quality under real constraints (mobile connectivity, synchronization, performance, and deployment readiness). The value of the project is measured by system completeness and user-facing utility.

The proposal is organized as follows: Section II presents stakeholder analysis and a comparison with available products, Section III describes system architecture, technologies, and implementation methods, Section IV summarizes expected and currently achieved results, and Section V concludes with practical impact and next development steps.

II Analysis of Stakeholders and Existing Solutions

The main stakeholders are small travel groups: friends, families, or colleagues who plan medium-complexity trips with several participants and a shared budget. In this scenario, each participant has different priorities (cost, activities, schedule flexibility), and the planning process is inherently collaborative. A second stakeholder group is trip owners or organizers, who need transparent control mechanisms (approval/rejection, member roles, and data consistency) without becoming a bottleneck.

Based on project requirements and comparative analysis, several stakeholder needs are central:

1. Single source of truth for trip data.
2. Transparent discussion and decision history.
3. Clear itinerary by dates and locations.
4. Expense tracking with understandable balances.
5. Minimal coordination overhead under unstable network conditions.

A market comparison confirms that partial tools exist, but integrated workflows remain limited. Splitwise is strong in debt balancing but does not provide a full collaborative itinerary and

idea lifecycle [1]. Wanderlog supports planning and collaboration, but shared budgeting, role-governed idea flow, and deeper integrated discussion patterns are not its primary focus [2]. TravelSpend is oriented toward expense logging rather than end-to-end collaborative trip design [3]. TripIt is effective for travel document aggregation and itinerary automation but is not designed as a collaborative planning workspace with approval and discussion loops [4].

Among domestic services, Yandex Travel has introduced a shared trip mode with invite links in the “My Trip” section, where participants can view bookings and recommendations [5]. However, this mode is read-oriented for participants: they cannot edit bookings or invite others, and the product scope is centered on transport/hotel booking workflows rather than full collaborative planning with expense balancing and moderated idea flow [5]. RUSSPASS is another Russian travel platform with mobile features focused on tours, places, and ticketed attractions [6]. Its public feature set emphasizes discovery and booking scenarios rather than a multi-user planning workspace with shared balances, itinerary editing governance, and real-time group discussion [6].

This project formalizes the gap through feature criteria: route planning, in-app comments, expense balancing, weather integration, and AI suggestions in one product. Existing products cover subsets of these criteria; the proposed application aims to integrate all of them into one Android-first workflow.

For a project thesis, this integrated scope is the key practical differentiator. The system is not trying to invent a new planning paradigm. Instead, it assembles proven components into an operational architecture where user effort and context switching are reduced. In other words, the expected advantage is workflow completeness and execution efficiency.

III Methods and Product Design

A. System architecture

The proposed application is implemented as a client-server system with clear separation of concerns. The Android client handles interaction, local persistence, and synchronization orchestration. The backend exposes domain APIs, enforces authorization and access rules, manages persistence, and integrates external providers. PostgreSQL is used for durable storage of domain entities, while migration scripts ensure versioned schema evolution [7], [8].

At the backend level, routing is modularized by domains (auth, users, trips, members, ideas, comments, itinerary, expenses, weather, AI, notifications, sync). This design keeps the codebase maintainable and supports incremental feature rollout. The server runtime is Kotlin + Ktor, selected for type-safe development, coroutine-friendly concurrency, and predictable HTTP/WebSocket support [9].

B. Android client design

The mobile app is implemented in Kotlin [10] with Jetpack Compose UI [11]. Dependency injection is handled via Hilt [12], and network communication is implemented through Retrofit [13]. Local session/configuration persistence uses DataStore, while image loading is handled by Coil.

The UI layer follows a screen/view-model pattern where each feature module has isolated state and event contracts. This modularity supports independent development of trip, idea, itinerary, expense, weather, and settings flows. Material Design 3 principles are applied for consistency and accessibility across key interfaces [14].

C. Data model and persistence

The backend data model covers the full trip planning lifecycle: users, trips, trip members, invite links, ideas, comments, itinerary days, activities, expenses, splits, weather snapshots, AI requests/suggestions, and notifications. Entity boundaries are designed to preserve collaborative semantics (for example, owner/member roles and idea approval states).

On the Android side, Room is used to persist synchronization state and pending changes [15]. This local layer is required for offline-first behavior and resilient user flows under intermittent connectivity.

D. Authentication, authorization, and access control

Authentication is based on Google token verification followed by application JWT issuance, which allows stateless request authorization for protected APIs [16]. The backend enforces role-aware checks for trip-level operations (membership, ownership, moderation actions). This protects collaborative resources from unauthorized mutation and aligns with the expected user trust model.

E. Collaboration flows and real-time communication

A critical practical requirement is immediate visibility of idea discussions for all trip participants. For this reason, the application implements WebSocket channels for comments. The backend broadcasts create/delete comment events, and clients update local discussion state in near real time. This removes polling overhead and supports a natural “live planning room” effect.

Invite links are generated server-side with expiry and revocation logic. This allows controlled onboarding into trip spaces and reduces organizer friction compared to manual member management.

F. Itinerary, expenses, and balancing logic

The itinerary subsystem supports trip-day structures and ordered activity lists. Users can assign ideas to days, reorder activities, edit details, and keep planning aligned with trip dates. The expense subsystem stores shared costs, split rules, and per-participant balance effects. From a stakeholder perspective, this replaces the common “separate spreadsheet + messenger confirmations” pattern with auditable in-app records.

G. Weather and AI integrations

Weather data is integrated through OpenWeather APIs with cache-aware refresh logic [17]. This avoids unnecessary external calls and provides predictable behavior in planning periods where forecast windows are limited.

AI suggestions are integrated through provider abstraction, with Yandex GPT as the production provider via the Yandex Foundation Models API [18]. This enables route/activity proposal generation from trip context and user-selected constraints. Importantly, AI is treated as an assistant, not an autonomous planner: generated suggestions can be reviewed and explicitly saved into the idea pipeline.

H. Offline synchronization strategy

Offline resilience is implemented through queued local changes and background synchronization jobs via WorkManager [19]. When the network is unavailable, edits are persisted locally and retried later. Once connectivity returns, the sync worker pushes pending changes and resolves applied/conflict statuses. This design is essential for mobile reliability and for real usage conditions with unstable connectivity.

I. Deployment and production orientation

The backend is configured for environment-based deployment and can be operated with standard service supervision, logging, and health endpoints. The production backend is deployed on Yandex.Cloud virtual infrastructure [20]. The Android app is configured with release packaging and App Links support for invite/deep-link flows. Distribution is planned through Google Play for production publication and updates [21].

Overall, the selected methods prioritize practical engineering goals: maintainability, predictable behavior under real network conditions, secure collaboration, and deployable architecture.

IV Anticipated and Achieved Results

From a project-thesis perspective, the expected result is a production-capable planning product that closes the workflow gap identified in Section II. The core acceptance criteria are:

1. End-to-end collaborative trip lifecycle in one app.
2. Stable role-based access control and invitation flow.
3. Real-time idea discussion.
4. Itinerary and shared expense management with transparent state.
5. Weather and AI assistance integrated into planning.
6. Offline editing with background synchronization.

At the current stage, the repository implementation already covers most of these modules in executable form (Android client, domain backend routes, persistence schema, migration scripts, synchronization worker, and WebSocket-based comments). This indicates strong progress from concept to integrated system. Remaining engineering focus is concentrated on final hardening:

broader automated test coverage, regression checks on synchronization edge cases, and release-quality polishing for publication.

In practical terms, even before full production rollout, the project demonstrates that integrated collaborative planning can be implemented as a coherent Android + Kotlin backend system rather than as loosely coupled tools.

V Conclusion

This project addresses a practical and recurring problem: fragmented group trip planning. The proposed system consolidates the key planning workflow into a single Android-centered product supported by a Kotlin Ktor backend and PostgreSQL persistence. The implemented architecture combines collaboration features (invites, comments, role checks), planning features (ideas, itinerary), finance features (expenses and balances), context features (weather), and assistance features (AI suggestions).

For a project-oriented thesis, the main contribution is engineering value: an integrated, deployable, and maintainable system that reduces context switching and coordination cost for real users. The work demonstrates how existing technologies can be combined into a product with clear user utility and realistic deployment path.

The next stage is operational maturity: expanding automated testing, refining synchronization conflict handling, and finalizing release procedures for Google Play. With these steps completed, the system can function not only as a graduation project artifact but as a usable digital service for collaborative travel planning.

Word Count: 1689

References

- [1] Splitwise, “Splitwise,” [Online]. Available: <https://www.splitwise.com/>. [Accessed: Feb. 14, 2026].
- [2] Wanderlog, “Wanderlog,” [Online]. Available: <https://wanderlog.com/>. [Accessed: Feb. 14, 2026].
- [3] TravelSpend, “TravelSpend,” [Online]. Available: <https://travel-spend.com/>. [Accessed: Feb. 14, 2026].
- [4] TripIt, “TripIt,” [Online]. Available: <https://www.tripit.com/>. [Accessed: Feb. 14, 2026].
- [5] Yandex, “Yandex Travel Simplifies Joint Trips,” Apr. 3, 2025. [Online]. Available: <https://yandex.ru/company/news/01-03-04-2025>. [Accessed: Feb. 14, 2026].
- [6] Google Play, “RUSSPASS,” [Online]. Available: <https://play.google.com/store/apps/details?id=ru.russpass.tourist>. [Accessed: Feb. 16, 2026].
- [7] PostgreSQL Global Development Group, “PostgreSQL Documentation,” [Online]. Available:

- <https://www.postgresql.org/docs/>. [Accessed: Feb. 14, 2026].
- [8] Redgate, “Flyway by Redgate Documentation,” [Online]. Available: <https://documentation.red-gate.com/flyway>. [Accessed: Feb. 16, 2026].
- [9] Ktor, “Ktor Documentation,” [Online]. Available: <https://ktor.io/docs/welcome.html>. [Accessed: Feb. 14, 2026].
- [10] JetBrains, “Kotlin Documentation,” [Online]. Available: <https://kotlinlang.org/docs/home.html>. [Accessed: Feb. 14, 2026].
- [11] Android Developers, “Jetpack Compose,” [Online]. Available: <https://developer.android.com/compose>. [Accessed: Feb. 14, 2026].
- [12] Android Developers, “Dependency Injection with Hilt,” [Online]. Available: <https://developer.android.com/training/dependency-injection/hilt-android>. [Accessed: Feb. 14, 2026].
- [13] Square, “Retrofit,” [Online]. Available: <https://square.github.io/retrofit/>. [Accessed: Feb. 14, 2026].
- [14] Google, “Material Design 3,” [Online]. Available: <https://m3.material.io/>. [Accessed: Feb. 14, 2026].
- [15] Android Developers, “Room Persistence Library,” [Online]. Available: <https://developer.android.com/training/data-storage/room>. [Accessed: Feb. 14, 2026].
- [16] Google, “Google Identity Services,” [Online]. Available: <https://developers.google.com/identity>. [Accessed: Feb. 14, 2026].
- [17] OpenWeather, “Weather API,” [Online]. Available: <https://openweathermap.org/api>. [Accessed: Feb. 14, 2026].
- [18] Yandex Cloud, “YandexGPT,” [Online]. Available: <https://yandex.cloud/en/docs/ai-studio/concepts/generation/models>. [Accessed: Feb. 16, 2026].
- [19] Android Developers, “WorkManager,” [Online]. Available: <https://developer.android.com/topic/libraries/architecture/workmanager>. [Accessed: Feb. 14, 2026].
- [20] Yandex Cloud, “Compute Cloud Documentation,” [Online]. Available: <https://yandex.cloud/en/docs/compute/>. [Accessed: Feb. 14, 2026].
- [21] Android Developers, “Distribute to Google Play,” [Online]. Available: <https://developer.android.com/distribute/google-play>. [Accessed: Feb. 14, 2026].